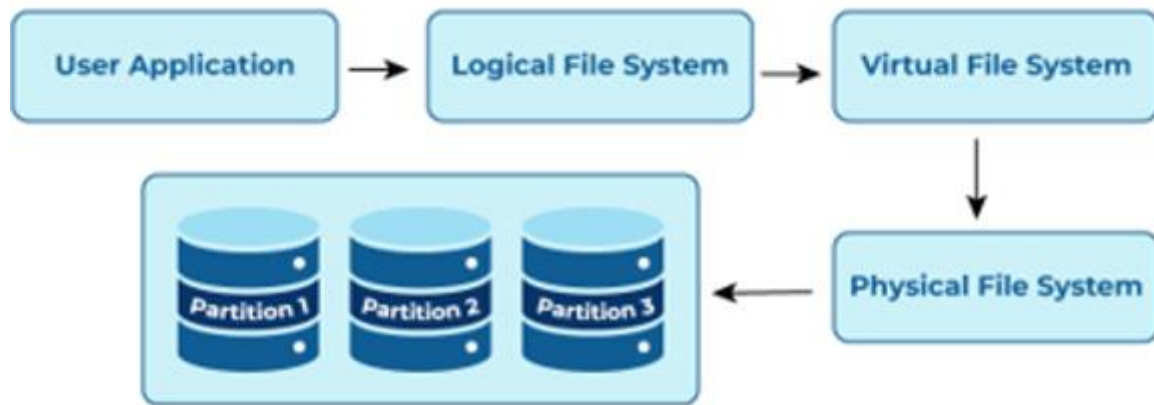


UNIT-V

File Systems

File systems are a crucial part of any operating system, providing a structured way to store, organize and manage data on storage devices such as hard drives, SSDs and USB drives.

The Architecture of a File System



File System

- **User Application:** The programs or software that request file operations like read, write, or delete.
- **Logical File System:** Manages metadata, file names, directories, and access permissions.
- **Virtual File System (VFS):** Acts as a bridge, allowing different file systems to work under a single interface.
- **Physical File System:** Handles the actual storage of data blocks on the disk.
- **Partition 1, Partition 2, Partition 3:** Divisions of the storage device where files are stored physically.

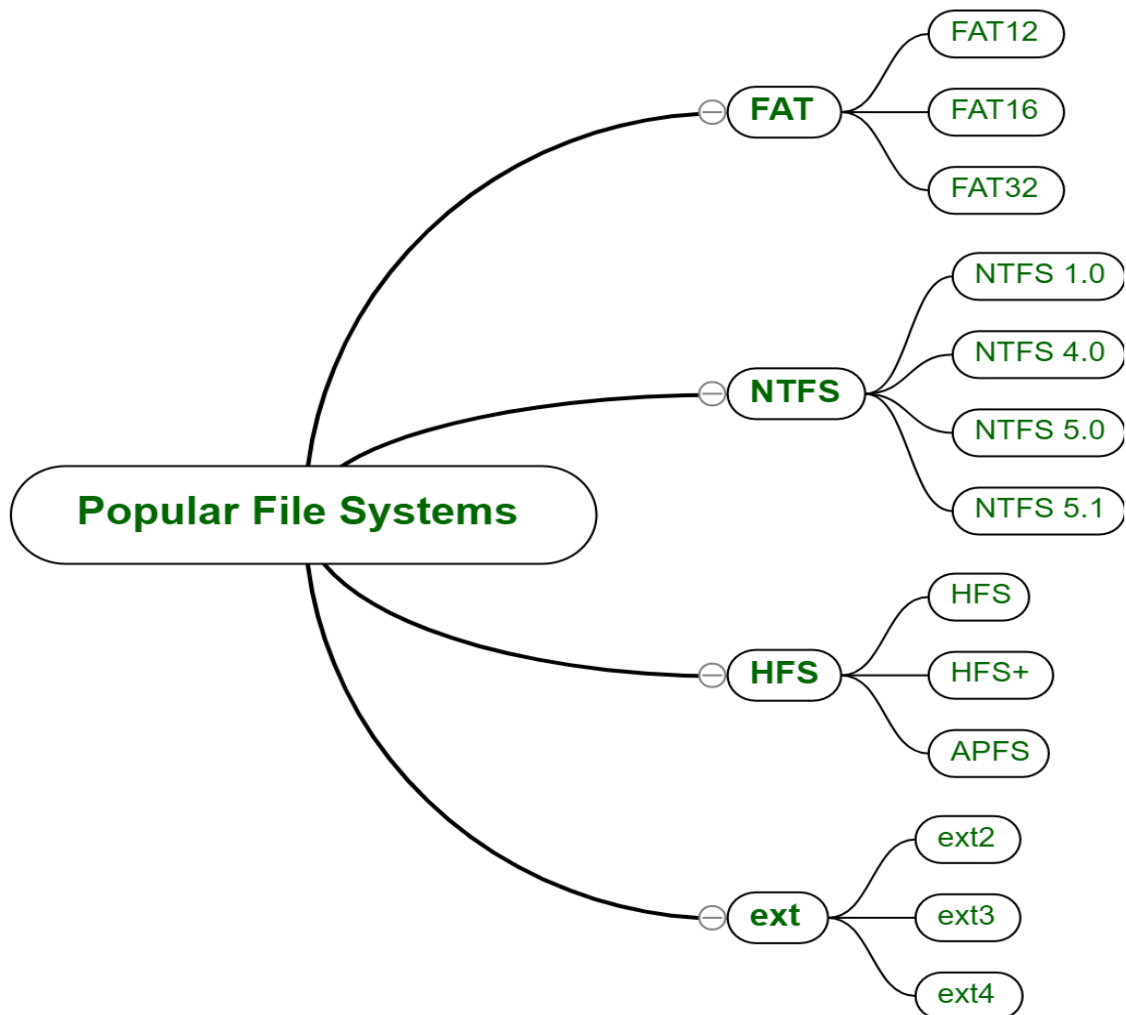
***Note:** It acts as a bridge between the operating system and the physical storage hardware, allowing users and applications to perform CRUD Operations on files in an organized and efficient manner.*

Popular File Systems

Some common types of file systems include:

- **FAT (File Allocation Table):** An older file system used by older versions of Windows and other operating systems.
- **NTFS (New Technology File System):** A modern file system used by Windows. It supports features such as file and folder permissions, compression and encryption.

- **ext (Extended File System):** A file system commonly used on Linux and Unix-based operating systems.
- **HFS (Hierarchical File System):** A file system used by macOS.
- **APFS (Apple File System):** A new file system introduced by Apple for their Macs and iOS devices.



Popular File Systems

The name of the file is divided into two parts as shown below:

1. Name
2. Extension, separated by a period.

Issues Handled by File System

- A free space is created on the hard drive whenever a file is deleted from it.
- To reallocate them to other files, many of these spaces may need to be recovered.
- Choosing where to store the files on the hard disc is the main issue with files one block may or may not be used to store a file.

File Directories

The collection of files is a file directory and contains information about the files, including attributes, location, etc. Much of this information, is managed by the operating system.

***Note:** The directory is itself a file, accessible by various file management routines.*

File Types and Their Content

File Type	Usual extension	Function
Executable	exe, com, bin	Read to run machine language program
Object	obj, o	Compiled, machine language not linked
Source Code	C, java, pas, asm, a	Source code in various languages
Batch	bat, sh	Commands to the command interpreter
Text	txt, doc	Textual data, documents
Word Processor	wp, tex, rrf, doc	Various word processor formats
Archive	arc, zip, tar	Related files grouped into one compressed file
Multimedia	mpeg, mov, rm	For containing audio/video information
Markup	xml, html, tex	It is the textual data and documents
Library	lib, a, so, dll	It contains libraries of routines for programmers
Print or View	gif, pdf, jpg	It is a format for printing or viewing an ASCII or binary file.

File Types & their Content

***Note:** It may be kept in the disk's non-contiguous blocks. We must keep track of all the blocks where the files are partially located.*

Advantages of Maintaining Directories

- **Efficiency:** A file can be located more quickly.
- **Naming:** It becomes convenient for users as two users can have same name for different files or may have different name for same file.
- **Grouping:** Logical grouping of files can be done by properties e.g. all java programs, all games etc.

Structures of Directory

- **Single-Level Directory:** In this, a single directory is maintained for all the users.
- **Two-Level Directory:** In these separate directories for each user is maintained.
- **Tree-Structured Directory:** The directory is maintained in the form of a tree. Searching is efficient and also there is grouping capability. We have absolute or relative path name for a file.

File Allocation Methods

There are several types of file allocation methods. These are mentioned below:

- Contiguous Allocation
- Linked Allocation (Non-contiguous allocation)
- Indexed Allocation

Disk Free Space Management

To perform any of the file allocation techniques, it is necessary to know what blocks on the disk are available. Thus, to manage unallocated disk space, two common methods are used:

- **Bit Tables:** This uses a vector containing one bit for each block on the disk. Each entry for a 0 corresponds to a free block and each 1 corresponds to a block in use.
- **Free Block List:** In this method, each block is assigned a number sequentially and the list of the numbers of all free blocks is maintained in a reserved block of the disk.
- **Linked List:** Free blocks are linked together in a list. Each free block stores the address of the next free block.
- **Boundary Tags:** Each block contains a boundary tag indicating its size and whether it is free or occupied.

File Allocation Methods

The allocation methods define how the files are stored in the disk blocks. There are three main disk space or file allocation methods.

- Contiguous Allocation
- Linked Allocation
- Indexed Allocation

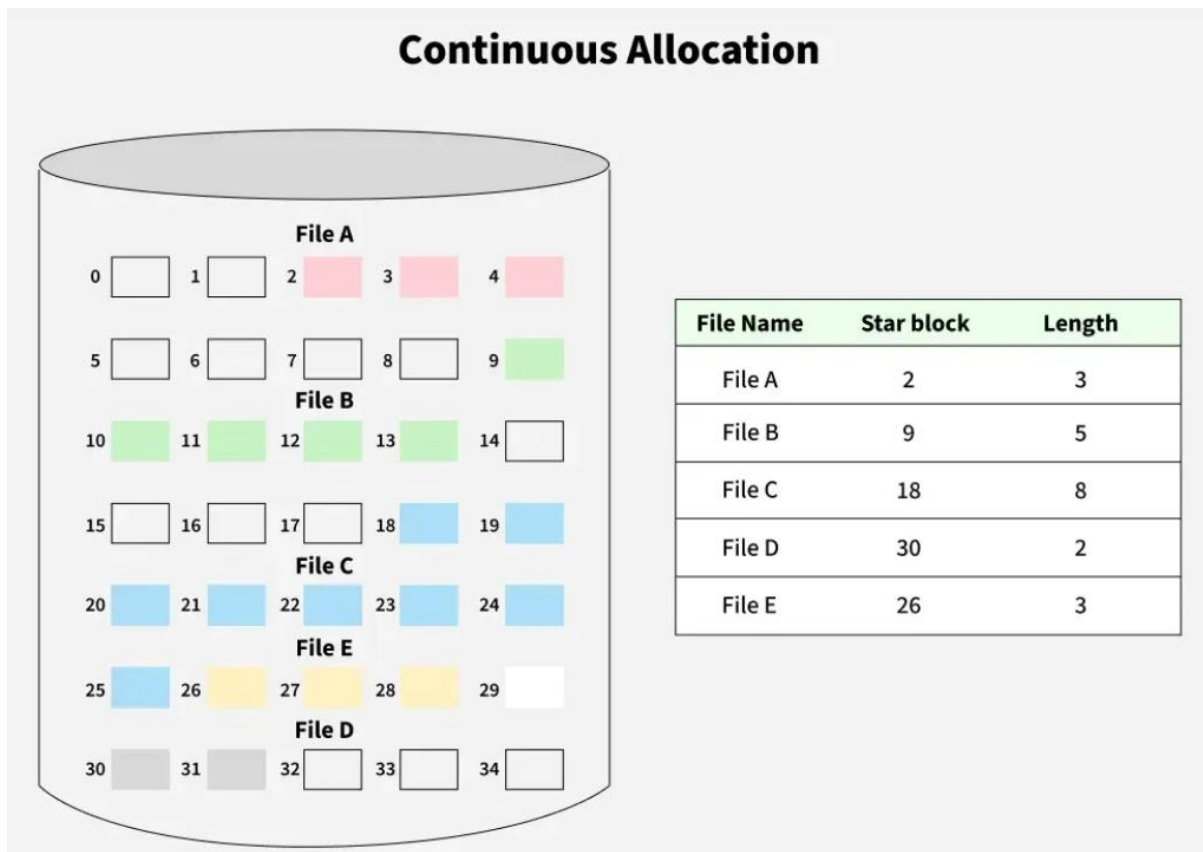
The main idea behind these methods is to provide Efficient disk space utilization & Fast access to the file blocks.

Contiguous Allocation

In this scheme, each file occupies a contiguous set of blocks on the disk. This means that given the starting block address and the length of the file (in terms of

blocks required), we can determine the blocks occupied by the file. The directory entry for a file with contiguous allocation contains:

- Address of starting block
- Length of the allocated portion.
- **Example:** If a file requires n blocks and is given a block b as the starting location, then the blocks assigned to the file will be: $b, b+1, b+2, \dots, b+n-1$.



Contiguous Allocation

Advantages

- Both the Sequential and Direct Accesses are supported by this. For direct access, the address of the k th block of the file which starts at block b can easily be obtained as $(b+k)$.
- This is extremely fast since the number of seeks are minimal because of contiguous allocation of file blocks.

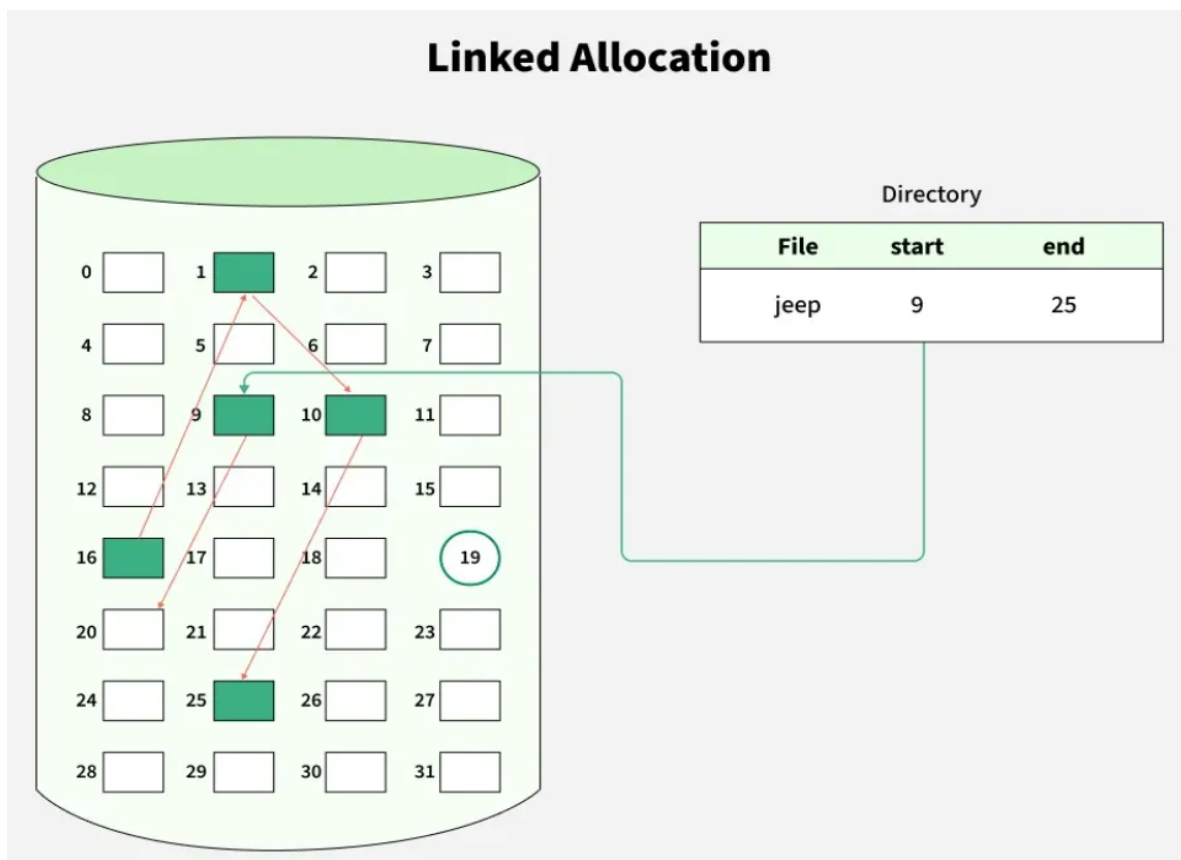
Disadvantages

- This method suffers from both internal and external fragmentation. This makes it inefficient in terms of memory utilization.
- Increasing file size is difficult because it depends on the availability of contiguous memory at a particular instance.

Linked Allocation

In this scheme, each file is a linked list of disk blocks which need not be contiguous. Here,

- The disk blocks can be scattered anywhere on the disk. The directory entry contains a pointer to the starting and the ending file block.
- Each block contains a pointer to the next block occupied by the file.
- **Example:** The file 'jeep' in following image shows how the blocks are randomly distributed. The last block (25) contains -1 indicating a null pointer and does not point to any other block.



Linked Allocation

Advantages

- This is very flexible in terms of file size. File size can be increased easily since the system does not have to look for a contiguous chunk of memory.
- This method does not suffer from external fragmentation. This makes it relatively better in terms of memory utilization.

Disadvantages

- Because the file blocks are distributed randomly on the disk, a large number of seeks are needed to access every block individually. This makes linked allocation slower.
- It does not support random or direct access. We can not directly access the blocks of a file. A block k of a file can be accessed by traversing k blocks sequentially (sequential access) from the starting block of the file via block pointers.
- Pointers required in the linked allocation incur some extra overhead.

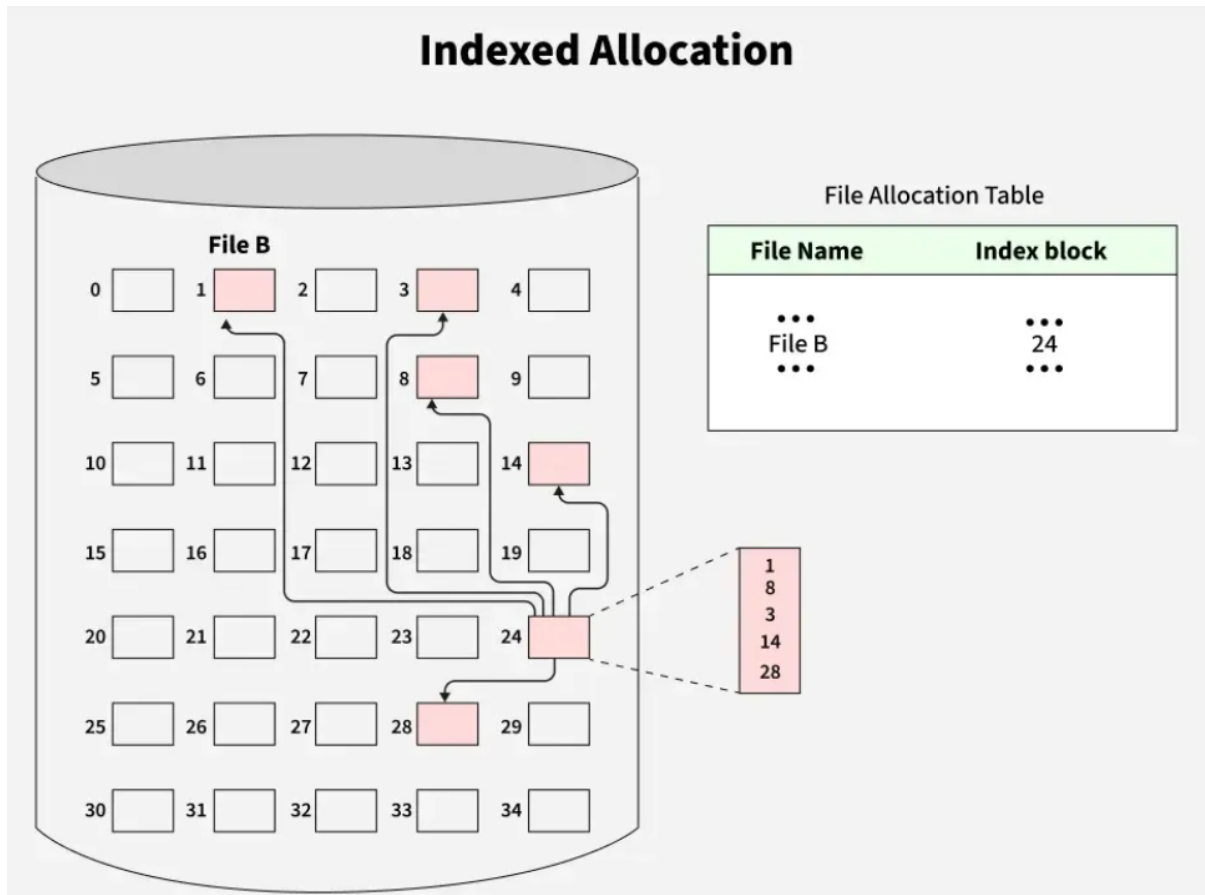
Indexed Allocation

In this scheme, a special block known as the Index block contains the pointers to all the blocks occupied by a file. Here,

- Each file has its own index block.
- The i^{th} entry in the index block contains the disk address of the i^{th} file block.
- The directory entry contains the address of the index block as shown in the image.

Advantages

- This supports direct access to the blocks occupied by the file and therefore provides fast access to the file blocks.
- It overcomes the problem of external fragmentation.



Indexed Allocation

Disadvantages

- The pointer overhead for indexed allocation is greater than linked allocation.
- For very small files, say files that expand only 2-3 blocks, the indexed allocation would keep one entire block (index block) for the pointers which is inefficient in terms of memory utilization. However, in linked allocation we lose the space of only 1 pointer per block.

Indexing Schemes for Large Files

When a file is very large, a single index block may not have enough space to hold all the pointers to the data blocks. To manage this, the following indexing mechanisms are used:

1. Linked Scheme

- In this approach, multiple index blocks are linked together.
- Each index block contains some pointers to disk blocks and a pointer to the next index block.
- The chain continues until all the pointers are covered.

2. Multilevel Index

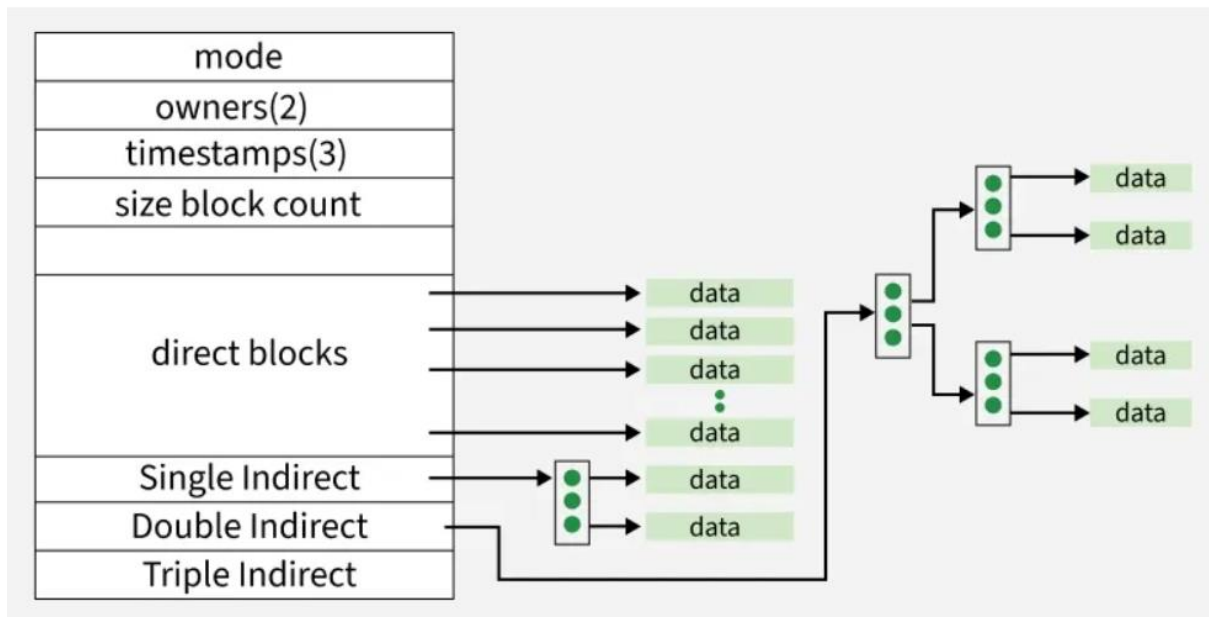
- Instead of a single large index block, we use a hierarchy of index blocks.
- The first-level index block points to second-level index blocks and those second-level blocks contain pointers to the actual data blocks.
- Can be extended to three or more levels as required by file size.

3. Combined Scheme (Inode Structure)

A special block called the Inode (Information Node) stores File metadata (name, size, permissions, timestamps) & Block addresses (pointers).

The Inode typically contains:

- 1. Direct Pointers:** Point directly to data blocks (e.g., first 10 pointers).
- 2. Single Indirect Pointer:** Points to a block that holds further block addresses.
- 3. Double Indirect Pointer:** Points to a block that contains pointers to other indirect blocks, which in turn point to data blocks.
- 4. Triple Indirect Pointer:** Adds another layer of indirection for extremely large files.



Combined Scheme (Inode Structure)

Why Combined Scheme Is Preferred:

- Combines direct access for small files (faster).
- Allows efficient scalability for large files.
- Widely used in modern filesystems like ext2, ext3, ext4 (Linux).